

El Ezaby - Mobile Pharmacy Shopping App

Platform: Flutter (Android / iOS)

Backend: Firebase (Auth, Firestore, Cloud Functions, Storage)

1. Executive Summary

El Ezaby is a cross-platform mobile shopping application modelled on Egypt's leading pharmacy chain of the same name. The app gives customers a single place to browse pharmacy and personal-care products, scan barcodes on physical items, manage a cart and delivery addresses, upload prescriptions, and earn loyalty rewards on every order. It is built with Flutter for the client and Firebase for all backend services, allowing a single codebase to ship to both Android and iOS while keeping infrastructure overhead minimal.

The application targets everyday consumers who want the convenience of home delivery for medicine and wellness products, combined with modern shopping conveniences such as offers, favorites, and a rewards program. From an engineering perspective, the project demonstrates a clean Provider-based state architecture, a fully typed data model layered on Firestore, and an emphasis on responsive UX, every Firebase call exposes explicit loading, error, and empty states.

This report walks through the app from five perspectives: product purpose and user experience, technical architecture, the Firebase data model and security posture, the standout features (barcode scanning, prescription upload, and the rewards system), and finally the engineering practices and rules the project follows to remain maintainable.

2. Product Overview and User Experience

El Ezaby is organised around five primary tabs accessed through a bottom navigation bar: Home, Shop, Scanner, Offers, and More. The Home tab serves as the landing experience, it surfaces featured categories, the user's reward progress, current promotions, and a search bar that routes into the full catalogue. The Shop tab presents the complete product taxonomy: top-level categories drill into subcategories and sub-subcategories, ending in a product list screen that supports filtering and sorting.

The Scanner tab opens a live camera view powered by the `mobile_scanner` plugin. Pointing the device at a product barcode either resolves it to an existing catalogue item (showing price, manufacturer, origin, and usage steps) or, if unknown, offers to send the scan as a query to the support team. A simulated AR overlay floats product information on top of the camera feed to make the scanner feel modern and visual without requiring a dedicated AR SDK.

The Offers tab aggregates all products flagged as on-promotion. The More tab is the user's account hub: profile, saved addresses, favorites, order history, notifications, and developer utilities such as seeding demo data.

Shopping flow itself is conventional: add to cart, view cart with a free-shipping progress card, check out by selecting one of the saved addresses, place the order, and land on a success screen that confirms the reward points earned. Prescription-required items can be purchased after uploading a prescription image, which is attached to the order for pharmacist review. Every async operation across these flows is wrapped in explicit loading, error, and empty UI states a project-wide rule so the user never sees a blank screen waiting for Firestore.

Visually the app uses a calm, medical-grade palette built around a primary blue (#0087C8) accented with light-blue surfaces (#EAF8FC, #D5F1FA), a red badge color (#FF3B30) for cart and notification counts, and a green (#20A766) reserved exclusively for the rewards program. The typography and component styling are centralised in a single AppTheme so the look and feel can be tuned in one place.

3. Technical Architecture

The codebase follows a feature-light, layered structure under lib/:

- main.dart initialises Firebase, then runApp.
- app.dart MaterialApp.router wired to GoRouter.
- core/constants/ AppColors, AppConstants.
- core/theme/ AppTheme.
- core/utils/ AppRouter (route table).
- models/ Plain Dart classes: AppUser, Product, Category, Subcategory, SubSubcategory, CartItem, AppOrder, Address.
- services/ Stateless wrappers around Firestore / Auth / Storage: AuthService, ProductService, CartService, OrderService, RewardService, AddressService, FavoritesService, NotificationService, SeedService.
- providers/ ChangeNotifier classes that hold UI state and orchestrate services: AuthProvider, ProductProvider, CartProvider, FavoritesProvider, AddressProvider, RewardProvider.
- screens/ One folder per screen group (auth, home, shop, product, cart, checkout, orders, scanner, offers, favorites, addresses, prescription, notifications, profile, more, splash).

- widgets/ Reusable building blocks: AppButton, AppSearchBar, AppBottomNav, GlobalAppBar, ProductCard, ProductImage, RewardProgressCard, FreeShippingCard.

State management is built on Provider rather than Riverpod or Bloc. Each provider exposes the user-facing state plus the actions a screen can invoke, and listens to Firestore snapshots for live updates (e.g. CartProvider streams carts/{uid}/items so quantity changes anywhere in the app reflect instantly in the bottom-nav badge).

Routing is handled by GoRouter, declared once in AppRouter, with the auth state from AuthProvider acting as a redirect guard between the splash, auth, and shell routes. The main shell hosts the bottom navigation so tab switches do not unmount the underlying state.

For images, every product or category thumbnail is rendered through CachedNetworkImage to avoid re-downloading assets and to provide consistent placeholder and error behaviour. Heavy work barcode decoding, image scaling is kept off the UI isolate; mobile_scanner runs its decoder in a native worker so the camera preview stays smooth.

4. Firebase Data Model and Backend

All persistent state lives in Firestore. The schema is deliberately flat and document-oriented:

- users/{uid} name, email, phone, rewardPoints, firstOrderCompleted, createdAt.
- products/{id} name, nameArabic, price, imageUrl, categoryId, barcode, manufacturer, origin, stock, rewardPoints, isOffer,

- usageSteps, description.
- categories/{id} name, emoji, sortOrder. Subcategory and sub-subcategory documents extend this to form a three-level taxonomy.
- carts/{uid}/items/{itemId} productId, quantity, price, addedAt.
- favorites/{uid}/items/{itemId} productId, addedAt.
- orders/{id} userId, items, subtotal, deliveryFee, total, status, rewardPointsEarned, createdAt.
- addresses/{uid}/items/{id} label, line1, city, governorate, phone, isDefault.

Authentication is handled by Firebase Auth and exposed through AuthService. The supported providers include email/password, Google Sign-In, and a guest mode that defers account creation until checkout. Because Google Sign-In on Android requires the SHA-1 fingerprint to be registered in the Firebase console, that step is part of the project's deployment checklist rather than something the app can self-configure.

Firestore security rules enforce that users can only read and write their own documents under carts, favorites, addresses, and orders, while products and categories are world-readable but admin-write-only. Rules are deployed with `firebase deploy --only firestore:rules`; any rule change requires a re-deploy before testing on a real client.

Cloud Functions are used for server-side concerns that should not run on the client: awarding reward points on order completion, sending order-status notifications, and validating prescription uploads. The project rule is strict: any function change must be followed by `firebase deploy`, only functions before the next debugging iteration, so the team never wastes time chasing a bug in code that is not actually running in production.

App Check is configured to block unauthenticated SDK calls. Debug tokens

are pre-registered for the emulator and developer devices so local testing is not silently blocked.

5. Key Features, Engineering Practices, and Roadmap

Barcode scanning. The Scanner tab uses `mobile_scanner` exclusively (no custom native camera code) and runs detection off the UI isolate. Seeded demo products carry barcodes prefixed with TEST- so the scanner can be exercised against printed test sheets during development. When an unknown code is detected, the user is offered a path to request that the product be added, turning failed scans into a data-collection signal rather than a dead end.

Prescription upload. Items flagged as prescription-required cannot be checked out until the user uploads a clear image of their prescription. Uploads go to Firebase Storage and are linked to the resulting order so a pharmacist can verify before dispatch. The upload screen explicitly handles compression and progress feedback so users on slow networks see motion rather than a frozen UI.

Rewards program. Every product carries a `rewardPoints` value, every completed order accumulates points on the user's profile, and a `RewardProgressCard` on Home shows how close the user is to the next tier. First-time orders trigger a one-off bonus tracked via the `firstOrderCompleted` flag, encouraging conversion. Points are awarded server-side by a Cloud Function to prevent client tampering.

Engineering rules. The project follows a small set of strict rules that keep the codebase healthy:

- flutter analyze must exit clean after every change
- zero warnings

tolerated.

- All user-facing strings use localization keys from the start (EN/DE ARB files); raw string literals in widgets are never acceptable.
- All async Firebase calls expose loading, error, and empty states.
- Generated files (freezed, json_serializable, riverpod_generator targets) must be regenerated with build_runner after any annotated-class change.
- Third-party package APIs are verified against current docs before use, not assumed from training data, versions move quickly in the Flutter ecosystem.
- Large goals are broken into committed checkpoints so a usage-limit interruption always leaves the repo in a mergeable state.

Roadmap. Natural next steps for the project include: a richer search experience with typo tolerance via Algolia or Typesense, push notifications for order status transitions, an in-app chat with a pharmacist, expanded payment options beyond cash on delivery, and proper Arabic RTL polish on the few screens that still assume left-to-right layout. Longer term, the simulated AR overlay on the scanner could be replaced with a real AR experience using ARCore / ARKit to highlight related products on a shelf.

Conclusion. El Ezaby is a focused, well-scoped Flutter + Firebase application that covers the full pharmacy-shopping journey: discover, scan, buy, track, and earn points without overreaching on architecture. The combination of a flat document model, Provider-based state, GoRouter navigation, and a disciplined engineering checklist gives the project a strong foundation for the roadmap items above.

6. Deployment and Distribution

To make the project trial-able by anyone with an Android phone, the app is published as a debug-signed APK hosted behind a small landing page on Firebase Hosting. The public URL is <https://elezaby-6cfd9.web.app>, and from there a single Download for Android button delivers the latest build directly to the device. A QR code on the same page lets visitors on a laptop scan with their phone camera and download without typing the URL. The page is intentionally minimal: it carries the elezaby logo, a short install walkthrough ("allow your browser to install unknown apps, open the APK, tap Install"), and a clearly worded disclaimer stating that this is a university project, not affiliated with or endorsed by the real El Ezaby Pharmacies chain.

The hosting layer is the same Firebase project (elezaby-6cfd9) that powers the app's backend, so a single firebase deploy cycle covers Firestore rules, Cloud Functions, and the public landing page. The hosting configuration adds a custom Content-Disposition header so the .apk is served as an attachment rather than rendered inline, which keeps the install flow predictable across Chrome, Samsung Internet, and Firefox.

The APK itself is produced with `flutter build apk --release` and ships as a single ~76 MB fat binary covering all common Android ABIs. Because the project does not yet have a release keystore, the build is signed with the local debug key, adequate for demo distribution but with two consequences worth noting: end users must keep "Install unknown apps" enabled for their browser, and any future update has to be produced from the same development machine (Android refuses to upgrade an installed package with one signed by a different key). Migrating to a proper release keystore is a small, well-understood next step when the project moves beyond the university demo audience.

Web and Play Store distribution were both considered and deliberately deferred. The web target is blocked today by two plugins that have no web implementation, `mobile_scanner` (barcode scanning) and `video_player` (used by the AR product previews); shipping a web build would mean

amputating the two most distinctive features. The Play Store route, while the most polished distribution channel, requires a one-time developer fee and a release-signed APK, and was judged out of scope for a university-grade deliverable that is meant to be evaluated and shared, not commercially distributed.

Backend deployment follows the same discipline as the rest of the project. Firestore rules and indexes are pushed with `firebase deploy --only firestore:rules,firestore:indexes` whenever the schema or access rules change, and Cloud Functions are redeployed with `firebase deploy --only functions` before any client-side debugging, the project rule of "never spend more than one iteration debugging without first confirming the latest code is actually deployed" applies here. The Authentication providers (Email/Password and Google Sign-In) are enabled in the Firebase Console, with the debug keystore's SHA-1 fingerprint registered so Google Sign-In works on installed devices.